

Prob 1. (40 pts) Assume the 16 1-BYTE values starting from address `0x2000_0000` are increasing numbers from 0, 1, 2, to 15. Consider the running of the following assembly code line-by-line sequentially:

```

1  ldr    r4, =0x20000000 @ (1)
2  ldr    r0, [r4, #4]    @ (2)
3  ldr    r1, [r4, #2]!   @ (3)
4  ldr    r2, [r4], #4    @ (4)
5  ldr    r3, [r4]        @ (5)

```

Part 1. (8 pts) Draw the memory map similar to that of Example 1 of Lctr 20 but with FOUR columns, each for a byte, Byte 0 to Byte 3, in a row. Each element should be a byte in TWO-digit hexadecimal. For example, if you have 11, you should write it as 0x0B.

Part 2. (32 pts total) Determine the following based on the memory map:

- (4 pts) the value of R4 after running (1).
- (8 pts) the values of R0 and R4 after running (2).
- (8 pts) the values of R1 and R4 after running (3).
- (8 pts) the value of R2 and R4 after running (4).
- (4 pts) the value of R3 after running (5).

Solution:

Part 1.

Address	Byte 0	Byte 1	Byte 2	Byte 3
0x2000_0000:	0x00	0x01	0x02	0x03
0x2000_0004:	0x04	0x05	0x06	0x07
0x2000_0008:	0x08	0x09	0x0A	0x0B
0x2000_000C:	0x0C	0x0D	0x0E	0x0F

Part 2.

- R4 = 0x2000_0000
- R0 = 0x0706_0504, R4 = 0x2000_0000
- R1 = 0x0504_0302, R4 = 0x2000_0002
- R2 = 0x0504_0302, R4 = 0x2000_0006
- R3 = 0x0908_0706

--

Explanation:

Part 1 is straight. Here, we only provide explanation of Part 2.

- Line (1), `ldr r4, =0x20000000`, loads a value directly to R4. So, R4's value is 0x2000_0000.
- Line (2) `ldr r0, [r4, #4]`, loads the word saved at 0x2000_0000 + 4 to R0 without changing the value of R4. So, R0 = 0x0706_0504, R4 = 0x2000_0000
- Line (3), `ldr r1, [r4, #2]!`, updates the value of R4 to 0x2000_0002 first and then load the word pointed by the new R4 to R1. So, R1 = 0x0504_0302, R4 = 0x2000_0002.
- Line (4), `ldr r2, [r4], #4`, load the word pointed by R4 to R2 first and then update the value of R4 to 0x2000_0006. So, R2 = 0x0504_0302, R4 = 0x2000_0006.
- Line (5), `ldr r3, [r4]`, load the word pointed by R4 to R3 without changing the value of R4. So, R3 = 0x0908_0706.

:-

Prob 2. (40 pts) Assume we are given code snippets below for two ASM functions, `mp_task1` and `mp_task2`:

1	<code>mp_task1:</code>		<code>mp_task2:</code>
2	<code>ldrsh r2, [r0], #4</code>		<code>ldrh r2, [r0, #2]!</code>
3	<code>str r2, [r1, #0]</code>		<code>str r2, [r1, #0]</code>
4	<code>ldrsh r3, [r0], #4</code>		<code>ldrh r3, [r0, #2]!</code>
5	<code>str r3, [r1, #4]</code>		<code>str r3, [r1, #4]</code>
6	<code>bx lr</code>		<code>bx lr</code>

Assume the above functions are called in the C program which has the following code

```
1 int16_t *p16s = arr_s; // arr_s, a signed input array defined somewhere
2 uint16_t *p16u = arr_u; // arr_u, an unsigned input array defined somewhere
3 int *p32 = arr_32; // arr_32 is an output array defined somewhere
```

Read the above code carefully and guess the functionalities of them. Then do the following:

- (20 pts, 10 each function) (a, 3 pts) Determine the number of parameters of the function; (b, 4 pts) Provide the prototypes of the above two functions in the C program; (c, 3 pts) Give a calling example. (Note that you can use some of the above defined variables when creating the calling examples. You can also use constants in the calling examples. If you need to see examples of calling examples, take a look at test code of the class demo project.)
- (20 pts, 10 each) Write the C code to finish the functionalities of the above two assembly functions. Note that the values of the above pointers may be changed during your operation. You need to reset them after each task if needed. For example, if you have changed the value of `p16s`, you can run `p16s = arr_s` again to reset `p16s` before use it again. (No need to write the C code in functions; just write C code in the after the last line of code where the pointers are defined in the given code above.)

Solution:

Part 1.

(3 pts each) Number of parameters of each function: 2 each, both are used as pointers.

(4 pts each) Prototypes:

```

1 void mp_task1(int16_t *pin, int32_t *pou);
2 void mp_task2(uint16_t *pin, int32_t *pou);

```

(3 pts each) Calling example in the C code:

```

1 mp_task1(p16s, p32);
2 mp_task2(p16u, p32);

```

Part 2.

```

1 int16_t *p16s = arr_s; // arr_s is an input array defined somewhere
2 uint16_t *p16u = arr_u; // arr_u is an input array defined somewhere
3 int *p32 = arr_32;      // arr_32 is an output array defined somewhere
4
5 // Task 1
6 *p32 = (int32_t)*p16s;
7 p16s += 2;
8 *(p32+1) = (int32_t)*p16s;
9 p16s += 2;
10
11 // Task 2
12 *p32 = (uint32_t)***p16u;
13 *(p32+1) = (uint32_t)***p16u;

```

:-

Explanation:

We focus on explanation of the assembly code below as once we understand the intention of the code, the solutions to the problems can follow easily.

For `mp_task1`.

- `ldrsh r2, [r0], #4` first loads a signed halfword from the current address pointed by R0 to R2. Hence, R0 has to be a pointer of `int16_t`. It then updates the value of R0 by 4, which means if we use a pointer, say `p16s`, we need to increment the pointer by TWO. So, the C code should look like

```

1 int16_t temp = *p16s;
2 p16s += 2;

```

- `str r2, [r1, #0]` saves the value held in `temp` to the address pointed by R1 without offset. As we see the store is STR, so we know R1 should be a pointer to a 32-bit number. If we use a pointer `p32`, the corresponding C code can be written as

```

1 *p32 = temp;

```

Combining the above two parts, we will have the C code as

```

1 *p32 = (int32_t)*p16s;
2 p16s += 2;

```

Note that we have added type-casting to avoid warning and clearly indicating our intention for the data types.

- The next two lines just repeat the first two with an offset for the output pointer. So, the corresponding code is below:

```
1 *(p32+1) = (int32_t)*p16s;
2 p16s += 2;
```

For `mp_task2`.

- `ldrh r2, [r0, #2]!` loads an unsigned halfword from pre-indexed address to R2. Hence, R0 has to be a pointer of `uint16_t`. Also, the value of R0 has to be pre-updated by 2, which means if we use a pointer, say `p16u`, we need to have the code like

```
1 uint16_t temp = *++p16u;
```

- `str r2, [r1, #0]` is the same as that for `mp_task1`. If we use a pointer, say `p32`, the corresponding C code can be written as

```
1 *p32 = temp;
```

Combining the above two parts, we will have the C code as

```
1 *p32 = (uint32_t)*++p16u;
```

Note that we have added type-casting to avoid warning and clearly indicating our intention for the data types.

- The next two lines just repeat the first two with an offset for the output pointer. So, the corresponding code is below:

```
1 *(p32+1) = (uint32_t)*p16s;
```

:-

Prob 3. (80 pts) Assume we are given code snippets below:

C code:

```
1 // Functions defined in a.s file
2 extern void mp_task1(uint8_t *pIpt, int32_t *pOup);
3 extern void mp_task2(uint8_t *pIpt, int32_t *pOup);
4 extern void mp_task3(uint8_t *pIpt, int32_t *pOup);
5 extern void mp_task4(uint8_t *pIpt, int32_t *pOup, int i);
6
7 // Global variable
8 uint8_t ipt[16]; // Input data
9 int32_t opt[4]; // Output data
10
11 int main(void) {
12     for (int i = 0; i < 16; i++) {
13         ipt[i] = i<<4;
14     }
15
16     mp_task1(ipt, opt);
```

```

17     printf("Out1 = 0x%X, Out2 = 0x%X\n", opt[0], opt[1]);
18
19     mp_task2(ipt, opt);
20     printf("Out1 = 0x%X, Out2 = 0x%X\n", opt[0], opt[1]);
21
22     mp_task3(ipt, opt);
23     printf("Out1 = 0x%X, Out2 = 0x%X\n", opt[0], opt[1]);
24
25     mp_task4(ipt, opt, 1);
26     printf("Out1 = 0x%X, Out2 = 0x%X\n", opt[0], opt[1]);
27
28     while (1);
29 }

```

Assembly code:

```

1  mp_task1:
2      ldr r2, [r0, #4]
3      str r2, [r1, #0]
4      ldr r2, [r0, #0xC]
5      str r2, [r1, #4]
6      bx  lr
7
8  mp_task2:
9      ldr r2, [r0, #4]!
10     str r2, [r1, #0]
11     ldr r2, [r0, #4]!
12     str r2, [r1, #4]
13     bx  lr
14
15  mp_task3:
16     ldr r2, [r0], #4
17     str r2, [r1, #0]
18     ldr r2, [r0], #4
19     str r2, [r1, #4]
20     bx  lr
21
22  mp_task4:
23     ldr r3, [r0, r2, lsl #2]
24     str r3, [r1, #0]
25     add r2, #1
26     ldr r3, [r0, r2, lsl #2]
27     str r3, [r1, #4]
28     add r2, #1
29     bx  lr

```

Draw the memory map similar to that of Example 1 of Lctr 20 but with FOUR columns, each for a byte, Byte 0 to Byte 3, in a row

You may have noticed that the ALL pointers in the assembly functions are pointers to 32-bit numbers although some of them are defined as pointers of 8-bit numbers. Please follow the assembly's intention.

Part 1. (8 pts) Draw the memory map similar to that of Example 1 of Lctr 20 but with FOUR columns, each for a byte, Byte 0 to Byte 3, in a row. Each element should be a byte in TWO-digit hexadecimal. For example, if you have 11, you should write it as 0x0B. Note that we are not given the specific address of `ipt`. The address of the memory can be labeled in a form based on `ipt`, such as `ipt + 0` and `ipt + 4`.

Part 2. Determine the following (without running the program first):

- (8 pts) the printout after running Task 1 (the 2-line of code for `mp_task1`).
- (16 pts) the printout after running Task 2 and the values in R0 and R1 in hexadecimal after running Task 2 assuming their values are 0x20001010 and 0x20001040, respectively, when the function is called.
- (16 pts) the printout after running Task 3 and the values in R0 and R1 in hexadecimal after running Task 3 assuming their values are 0x20001010 and 0x20001040, respectively, when the function is called.
- (16 pts) the printout after running Task 4 and the values in R2 and R3 in hexadecimal after running Task 4.
- (16 pts, 4 pts each) the corresponding C code for each task.

Soln:

Part 1. The memory map is

Address	Byte 0	Byte 1	Byte 2	Byte 3
<code>ipt + 0x00:</code>	0x00	0x10	0x20	0x30
<code>ipt + 0x04:</code>	0x40	0x50	0x60	0x70
<code>ipt + 0x08:</code>	0x80	0x90	0xA0	0xB0
<code>ipt + 0x0C:</code>	0xC0	0xD0	0xE0	0xF0

Part 2.

- 0x7060_5040, 0xF0E0_D0C0
- 0x7060_5040, 0xB0A0_9080. R0 = 0x2000_1018, R1 = 0x2000_1040
- 0x3020_1000, 0x7060_5040. R0 = 0x2000_1018, R1 = 0x2000_1040
- 0x7060_5040, 0xB0A0_9080. R2 = 0x03, R3 = 0xB0A0_9080
- The solution is not unique:

```

1 // Task 1
2 *opt = *((uint32_t *) (ipt + 4));
3 *(opt+1) = *((uint32_t *) (ipt + 12));
4
5 // Task 2
6 uint32_t *ipt32 = (uint32_t *) ipt;
7 *opt = *++ipt32;
8 *(opt+1) = *++ipt32;
9
10 // Task 3
11 uint32_t *ipt32 = (uint32_t *) ipt;
12 *opt = *ipt32++;
13 *(opt+1) = *ipt32++;
14
15 // Task 4
16 uint32_t *ipt32 = (uint32_t *) ipt;
```

```

17 *opt = *(ipt32 + i++);
18 *(opt+1) = *(ipt32 + i++);

```

```

-:-

```

Explanation:

We focus on explanation of the assembly code below as once we understand the intention of the code, the solutions to the problems can follow easily.

For `mp_task1`.

```

1 mp_task1:
2     ldr r2, [r0, #4]
3     str r2, [r1, #0]
4     ldr r2, [r0, #0xC]
5     str r2, [r1, #4]
6     bx lr

```

- `ldr r2, [r0, #4]` first loads a word from the current address pointed by R0 with an offset of 4 to R2. So, if we use `ipt`, the pointer to 8-bit numbers, the C code should look like

```

1 int32_t temp = *((uint32_t *) (ipt + 4));

```

Note that we have added type-casting to read a word, not a byte.

- `str r2, [r1, #0]` saves the value held in `temp` to the address pointed by R1 without offset. As we see the store is STR, so we know R1 should be a pointer to a 32-bit number. If we use a pointer `opt`, the corresponding C code can be written as

```

1 *opt = temp;

```

Combining the above two parts, we will have the C code as

```

1 *opt = *((uint32_t *) (ipt + 4));

```

- The next two lines just repeat the first two with appropriate offsets. So, the corresponding code is below:

```

1 *opt = *((uint32_t *) (ipt + 4));
2 *(opt+1) = *((uint32_t *) (ipt + 12));

```

Note that we can define a new pointer as we show below.

For `mp_task2`.

```

1 mp_task2:
2     ldr r2, [r0, #4]!
3     str r2, [r1, #0]
4     ldr r2, [r0, #4]!
5     str r2, [r1, #4]
6     bx lr

```

- `ldr r2, [r0, #4]!` loads a word from pre-indexed address to R2. So, if we use `ipt`, the pointer to 8-bit numbers, the C code should look like

```

1  ipt += 4;
2  int32_t temp = *(uint32_t *)ipt;

```

This can be improved by defining a pointer to 32-bit numbers, which will lead to cleaner code:

```

1  uint32_t *ipt32 = (uint32_t *) ipt;
2  int32_t temp = **ipt32;

```

- `str r2, [r1, #0]` is the same as that for `mp_task1`. If we use a pointer to 32-bit numbers, say `opt`, the corresponding C code can be written as

```

1  *opt = temp;

```

Combining the above two parts, we will have the C code as

```

1  uint32_t *ipt32 = (uint32_t *) ipt;
2  *opt = **ipt32;

```

- The next two lines just repeat the first two with an offset for the output pointer. So, the corresponding code is below:

```

1  *(opt+1) = **ipt32;

```

For `mp_task3`.

```

1  mp_task3:
2      ldr  r2, [r0], #4
3      str  r2, [r1, #0]
4      ldr  r2, [r0], #4
5      str  r2, [r1, #4]
6      bx  lr

```

The code is the same as for `mp_task2` except the pre-indexed addressing is changed to the post-indexed addressing.

For `mp_task4`.

```

1  mp_task4:
2      ldr  r3, [r0, r2, lsl #2]
3      str  r3, [r1, #0]
4      add  r2, #1
5      ldr  r3, [r0, r2, lsl #2]
6      str  r3, [r1, #4]
7      add  r2, #1
8      bx  lr

```

- `ldr r3, [r0, r2, lsl #2]` loads word from register-offset address to R3. Here, we know the following:
 - R0 is used as a pointer to 32-bit numbers.
 - R2 is used to hold the variable used as the index. As we use `r2, lsl #2`, we know that each it just changes to something like `*(ptr + i)`.

- The value of R0 is not changed.

If we use `i` as the index variable, the C code can be:

```
1 uint32_t *ipt32 = (uint32_t *) ipt;
2 int32_t temp = *(ipt32 + i);
```

- `str r2, [r1, #0]` is the same as that for `mp_task1`. If we use a pointer to 32-bit numbers, say `opt`, the corresponding C code can be written as

```
1 *opt = temp;
```

Combining the above two parts, we will have the C code as

```
1 uint32_t *ipt32 = (uint32_t *) ipt;
2 *opt = *(ipt32 + i);
```

- `add r2, #1` increment the value of `i` by one. This is done after all the above two operations. So, combining everything from this step and the previous ones, we have

```
1 uint32_t *ipt32 = (uint32_t *) ipt;
2 *opt = *(ipt32 + i++);
```

- The next three lines just repeat the first three with an offset for the output pointer. So, the corresponding code is below:

```
1 *(opt+1) = *(ipt32 + i++);
```

:-